

views::take_before

- - Abstract
 - Revision history
 - Discussion
 - Design
 - Implementation experience
 - Proposed change
 - References

Abstract

This paper proposes the Tier 1 adaptor in [P2760](#): `views::delimit`, which is renamed into `views::take_before` along with its corresponding view class to improve the C++29 ranges facilities.

Revision history

R0

Initial revision.

R1

Rename `views::delimit` to `views::take_before` based on feedback from Sofia SG9.

Extend conditional borrowed ranges for `views::take_before(p, '\0')`.

R2

Extend borrowed when value type satisfies *tidy-obj* based on feedback from Kona G9.

Discussion

`views::take_before` accepts a range (or iterator) and a specified value and produces a range ending with the first occurrence of that value. One common usage is to construct a NTBS range without calculating its actual length, for example:

```
const char* p = /* from elsewhere */;
const auto ntbs = views::take_before(p, '\0');
```

In other words, it is somewhat similar to the value-comparison version of `views::take_while`, which was originally classified into the *take/drop* family in [P2214](#). Following the schedule of C++23 high-priority adaptors, `views::take_before` has now been moved from Tier 2 to Tier 1, as it provides an intuitive way to treat an element with a specific value as a sentinel.

Design

Why not just `r | views::take_while(...)`?

`r | views::take_before(v)` is basically equivalent to `r | views::take_while([v](const auto& e) { return e != v; })`, which seems that it is sufficient to just implement it via `views::take_while`. However, doing so is not an appropriate choice for the following reasons.

First, it's unclear whether to save `v` in a lambda or use `bind_front(ranges::not_equal_to, v)` for a more general purpose, which introduces extra indirection anyway. In other words, if we only want to compare `*it == v`, it will be done indirectly by `invoke(bind_front(ranges::not_equal_to, v), *it) -> ranges::not_equal_to(v, *it) -> *it == v` in case of using `views::take_while`. As a result, we inevitably pay the cost of two extra function calls, which may bring performance issues because it is unrealistic to assume that the compiler can optimize out these calls in *any* case.

Secondly, additional specific constraints need to be introduced. Take lambda as an example, we should ensure that `[v = forward<V>(v)](const auto& e) -> bool { return e != v; }` can be constructed, but we cannot just put it in the *requires*-clause because capture list can produce hard errors, and the return statement also requires corresponding constraint. This indicates that at least `constructible_from<decay_t<V>, V>` and `equality_comparable_with<range_reference_t<R>, V>` need to be added to the adaptor's call operator, such a lengthy constraint seems bloated and ugly. Note that this is also true when using `bind_front`, as it is not a constraint function either.

Instead, if a new `take_before_view` class is introduced, we can just check whether `take_before_view(E, F)` is well-formed because the class already has the correct constraints.

Finally, if the bottom layer of `views::take_before` is `take_while_view`, then the former will have a `pred()` member which returns the *internal* lambda. This can lead to untold confusion, as the user passes in a value, but all he gets back is an unspecified predicate, and there's really not much use in getting such an unspecified object.

To sum up, it is necessary to introduce a new `take_before_view` class which is not that complicated.

What are the constraints for `take_before_view`?

The implementation of `take_before_view` is very similar to `take_while_view`. The difference is that we need to save a value instead of a predicate, and its sentinel needs to compare the current element with the value instead of invoking the predicate.

First, the value type needs to model `move_constructible` to be wrapped by `movable-box`:

```
template<view V, move_constructible T>
    requires input_range<V> && is_object_v<T> && /* */
    class take_before_view : public view_interface<take_before_view<V, T>> {
    [...]
    };
```

We also need to require the `*it == v` is well-formed. In range/v3 this part is spelled as `equality_comparable_with<V, range_reference_t<R>>` which is straightforward.

However, the author prefers to use `indirect_binary_predicate<ranges::equal_to, iterator_t<V>, const T*>` to further comply with the mathematical sound for cross-type equality comparison, even if this require that `range_value_t<R>` needs to be comparable with `V` which does not involve in the implementation.

This makes the constraints of `take_before_view` corresponding to `take_while_view` consistent with `ranges::find` corresponding to `ranges::find_if`.

Should we provide in-place constructors for `take_before_view`?

Nope.

`take_before_view` takes ownership of the value which might be more efficient to construct it via in-place construction, so it seems reasonable to provide such a constructor like `take_before_view(V base, in_place_t, Args&&... args)`.

However, unlike `single_view/repeat_view`, in this case, we also need to explicitly specify the type of `V` for `take_before_view`, which was actually deduced by CTAD before. It turns out that introducing such a constructor does not have much usability.

Should we support the iterator version of `views::take_before`?

range/v3's `views::take_before` also supports accepting an iterator `it` that returns `subrange(it, unreachable_sentinel) | views::take_before(v)`. Providing such overloading does have a certain value, because sometimes we may only have `const char*`.

Do we need to introduce `views::c_str` as well?

`views::c_str` is also classified in T1 in [P2760](#), which allows us to view a null-terminated character array as a range. It produces a sized `subrange<const char*, const char*>` when taking `const char (&)[N]` such as string literals, and `views::take_before(p, '\0')` when taking a character pointer `p`.

Since the former can be easily achieved through `views::all("hello")` or `subrange("hello")`, and the latter is just another way of writing `views::take_before('\0')`, the author believes that the value it provides is limited, so it is not introduced in this paper. However, the author remains neutral.

`views::take_before` vs `views::take_until`?

At the Sofia meeting, SG9 agreed that `take_before` is a more appropriate name for this adaptor, as the semantics of *before* are clear and intuitively indicate that the resulting range ends prior to the specified value. While an alternative name such as `take_until` was considered, it introduces potential ambiguity regarding whether the terminating value is included or excluded. Such ambiguity is undesirable, particularly in the context of C++ Ranges, which places a strong emphasis on clarity, consistency, and semantic predictability in naming. The author concurs with this assessment and prefers the name `take_before`.

Should `views::take_before(p, '\0')` be a borrowed range?

In R0, the specified value is always stored in `take_before_view` with its sentinel storing a pointer to `take_before_view` for accessing the value, which makes `take_before_view` never a borrowed range.

During Sofia, SG9 raised concerns that this might not be desirable since such non-borrowability would inconvenience users.

For example, for trivial values such as integers, we can store them in the sentinel so that `views::take_before(p, '\0')` or `views::take_before(r, 42)` can be a borrowed range, which also eliminates indirect access to the value when checking for end.

However, during Kona, SG9 preferred to use the implementation-only trait consistent with [P3117](#) to extend borrowability to allow `views::take_before(r, cw<42>)`, although this made the previous example no longer borrowable, and it also meant that the sentinel class no longer needed to store the value, which could lead to better performance.

Implementation experience

The author implemented `views::take_before` based on libc++, see [here](#).

Proposed change

This wording is relative to [N5014](#).

1. Add a new feature-test macro to 17.3.2 [\[version.syn\]](#):

```
#define cpp_lib_ranges_take_before_2026XXL // freestanding, also in <ranges>
```

2. Modify 25.2 [\[ranges.syn\]](#), Header `<ranges>` synopsis, as indicated:

```
// mostly freestanding
#include <compare> // see \[compare.syn\]
```

```

#include <initializer_list>    // see \[initializer.list.syn\]
#include <iterator>           // see \[iterator.synopsis\]

namespace std::ranges {
    [...]
    namespace views { inline constexpr unspecified drop_while = unspecified; }

    \[\[range.take.before\], take before view
    template<class T>
    constexpr bool tidy_obj = // exposition only
    is_empty_v<T> &&
    is_trivially_default_constructible_v<T> &&
    is_trivially_destructible_v<T>;

    template<view V, move_constructible T>
    requires input_range<V> && is_object_v<T> &&
    indirect_binary_predicate<ranges::equal_to, iterator_t<V>, const T*>
    class take_before_view;

    template<class V, class T>
    constexpr bool enable_borrowed_range<take_before_view<V, T>> =
    enable_borrowed_range<V> && tidy_obj<T>;

    namespace views { inline constexpr unspecified take_before = unspecified; }
    [...]
}

```

3. Add **26.7.2 Take before view** [[range.take.before](#)] after 25.7.13.1 [[range.drop_while](#)] as indicated:

[26.7.2.1] Overview [[range.take.before.overview](#)]

-1- Given a specified value `val` and a view `r`, `take_before_view` produces a view of the range [`ranges::begin(r)`, `ranges::find(r, val)`).

-2- The name `views::take_before` denotes a customization point object ([\[customization.point.object\]](#)). Let `E` and `F` be expressions, and let `T` be `remove_cvref_t<decltype(E)>`. The expression `views::take_before(E, F)` is expression-equivalent to `take_before_view(subrange(E, unreachable_sentinel), F)` if `T` models `input_iterator` and does not model `range`; otherwise, `take_before_view(E, F)`, in which case `views::take_before` denotes a range adaptor object ([\[range.adaptor.object\]](#)).

-3- [Example 1:

```

const char* one_two = "One?Two";
for (auto c : views::take_before(one_two, '?')) {
    cout << c;           // prints One
}

```

— end example]

[26.7.2.2] Class template `take_before_view` [[range.take.before.view](#)]

```

namespace std::ranges {
    template<view V, move_constructible T>
    requires input_range<V> && is_object_v<T> &&
    indirect_binary_predicate<ranges::equal_to, iterator_t<V>, const T*>
    class take_before_view : public view_interface<take_before_view<V, T>> {
    // \[range.take.before.sentinel\], class template take_before_view::sentinel
    template<bool> class sentinel; // exposition only

    V base_ = V(); // exposition only
    movable_box<T> value_; // exposition only

    public:
    take_before_view() requires default_initializable<V> && default_initializable<T> = default;
    constexpr explicit take_before_view(V base, const T& value) requires copy_constructible<T>;
    constexpr explicit take_before_view(V base, T& value);

    constexpr V base() const & requires copy_constructible<V> { return base_; }
    constexpr V base() && { return std::move(base_); }

    constexpr auto begin() requires (!simple_view<V>)
    { return ranges::begin(base_); }

    constexpr auto begin() const
    requires range<const V> &&
    indirect_binary_predicate<ranges::equal_to, iterator_t<const V>, const T*>
    { return ranges::begin(base_); }

    constexpr auto end() requires (!simple_view<V>) {
    if constexpr (tidy_obj<T>)
        return sentinel<false>(ranges::end(base_));
    else
        return sentinel<false>(ranges::end(base_), addressof(*value_));
    }

    constexpr auto end() const
    requires range<const V> &&
    indirect_binary_predicate<ranges::equal_to, iterator_t<const V>, const T*> {
    if constexpr (tidy_obj<T>)
        return sentinel<true>(ranges::end(base_));
    else
        return sentinel<true>(ranges::end(base_), addressof(*value_));
    }
};

template<class R, class T>

```

```

    take_before_view(R&&, T) -> take_before_view<views::all_t<R>, T>;
}

constexpr explicit take_before_view(V base, const T& value) requires copy_constructible<T>;

-1- Effects: Initializes base_ with std::move(base) and value_ with value.

constexpr explicit take_before_view(V base, T&& value);

-2- Effects: Initializes base_ with std::move(base) and value_ with std::move(value).

[26.7.7.3] Class template take_before_view::sentinel [range.take.before.sentinel]

namespace std::ranges {
    template<view V, move_constructible T>
        requires input_range<V> && is_object_v<T> &&
            indirect_binary_predicate<ranges::equal_to, iterator_t<V>, const T*>
    template<bool Const>
    class take_before_view<V, T::sentinel {
        using Base = maybe_const<Const, V>;                                // exposition only

        sentinel_t<Base> end_ = sentinel_t<Base>();                      // exposition only
        const T* value_ = nullptr;                                       // exposition only, present only
                                                                    // if tidy-obj<T> is false

        constexpr sentinel(sentinel_t<Base> end, const T* value)
            requires (!tidy-obj<T>);                                     // exposition only
        constexpr sentinel(sentinel_t<Base> end)
            requires tidy-obj<T>;                                       // exposition only

    public:
        sentinel() = default;
        constexpr sentinel(sentinel_t<Const> s)
            requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

        constexpr sentinel_t<Base> base() const { return end_; }

        friend constexpr bool operator==(const iterator_t<Base>& x, const sentinel& y);

        template<bool OtherConst = !Const>
            requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
        friend constexpr bool operator==(const iterator_t<maybe-const<OtherConst, V>>& x,
                                         const sentinel& y);
    };
}

constexpr sentinel(sentinel_t<Base> end, const T* value) requires (!tidy-obj<T>);

-1- Effects: Initializes end_ with end and value_ with value.

constexpr sentinel(sentinel_t<Base> end) requires tidy-obj<T>;

-2- Effects: Initializes end_ with end.

constexpr sentinel(sentinel_t<Const> s)
    requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

-3- Effects: Initializes end_ with std::move(s.end_) and value_ with s.value_ if value_ is present.

friend constexpr bool operator==(const iterator_t<Base>& x, const sentinel& y);

template<bool OtherConst = !Const>
    requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr bool operator==(const iterator_t<maybe-const<OtherConst, V>>& x,
                                const sentinel& y);

-4- Returns: If value_ is present, y.end_ == x || *y.value == *x; otherwise, y.end_ == x || T() == *x.

```

References

[P2760R1]

Barry Revzin. A Plan for C++26 Ranges. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2760r1.html>

[P3117R1]

Zach Laine. Extending Conditionally Borrowed. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3117r1.html>